Proyecto 1: Docker con dos contenedores comunicados

Documento de evidencias – Entrega proyecto 1 (Escenario 3)

Estudiante: Santiago Gamboa

Profesor: John Olarte

Asignatura: Integración continua

Repositorio: https://github.com/santiagogamboac/CICDSFT2

10 de noviembre de 2026

1. Introducción

El objetivo del proyecto es construir dos contenedores Docker que se comuniquen entre sí dentro de una red propia, orquestados con Docker Compose e integrados a un pipeline de integración continua en GitHub Actions. Se desarrolló una aplicación de notas: el **contenedor 1 (app)** es una aplicación web en Node.js/Express que ofrece un frontend y una API REST; el **contenedor 2 (db)** es una base de datos PostgreSQL que almacena las notas en un volumen persistente.

2. Arquitectura

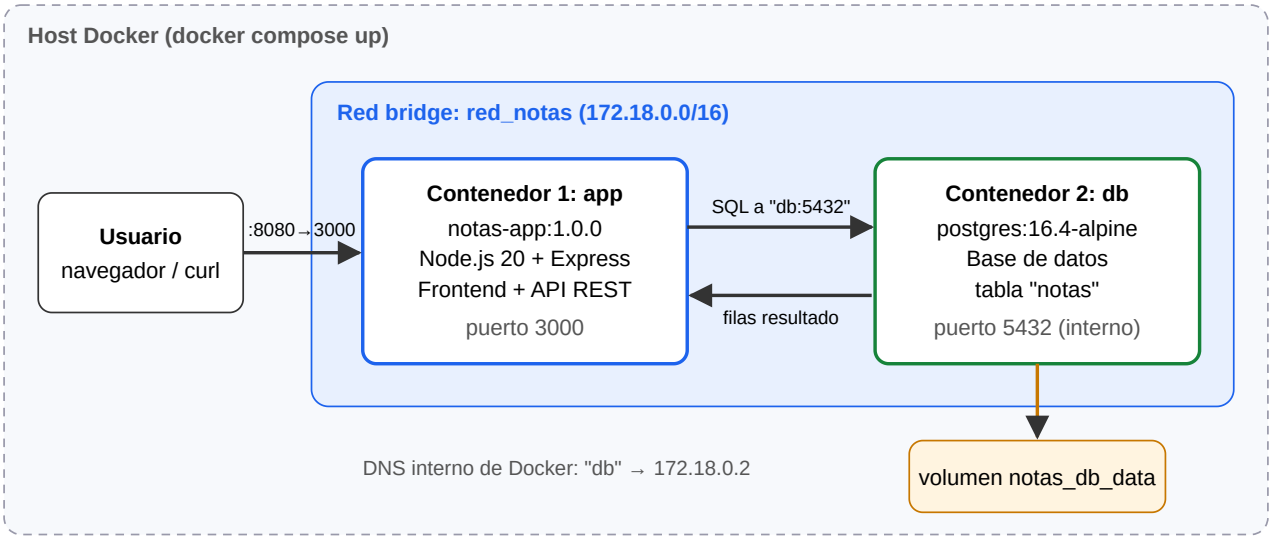


Figura 1. Diagrama de arquitectura. El usuario accede a la app por el puerto 8080 del host; la app se conecta a la base de datos usando el nombre del servicio `db`, que el DNS interno de Docker resuelve dentro de la red `red_notas`. La base de datos no expone puertos al host.

Requisito	Implementación
Repositorio	Carpetas `app/` y `db/` por servicio, README con comandos, commits por etapa.
Contenedor 1	`app/Dockerfile` con `node:20.18-alpine` (versión fija), `npm ci`, `EXPOSE 3000`, `CMD ["node","server.js"]`, usuario sin privilegios.
Contenedor 2	`postgres:16.4-alpine` configurado con variables `POSTGRES_USER/PASSWORD/DB`, volumen `notas_db_data` y script `init.sql`.
Docker Compose	Dos servicios, red `red_notas`, puertos, variables, `depends_on` con `service_healthy` y healthchecks.
Comunicación	`DB_HOST=db` (nombre del servicio). Prueba de guardar y leer una nota.

Seguridad	`.dockerignore`, `.env` excluido en `.gitignore`, `.env.example` en el repositorio, sin secretos en el código, 0 vulnerabilidades en `npm audit`.
CI	`.github/workflows/docker-ci.yml`: construye las imágenes, levanta los contenedores y prueba la comunicación en cada push y pull request.

3. Archivos principales

3.1 app/Dockerfile

```
# Contenedor 1: aplicación web (Node.js + Express)
# Imagen base oficial con versión fija (no "latest")
FROM node:20.18-alpine

WORKDIR /usr/src/app

# Instalar dependencias primero para aprovechar la caché de capas
COPY package.json package-lock.json ./
RUN npm ci --omit=dev && npm cache clean --force

# Copiar el código de la aplicación
COPY server.js ./
COPY public ./public

# Ejecutar como usuario sin privilegios (incluido en la imagen oficial)
USER node

ENV APP_PORT=3000
EXPOSE 3000

CMD ["node", "server.js"]
```

3.2 docker-compose.yml

```
services:
  # Contenedor 2: base de datos PostgreSQL
  db:
    image: postgres:16.4-alpine
    container_name: notas-db
    restart: unless-stopped
    environment:
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      POSTGRES_DB: ${POSTGRES_DB}
    volumes:
      - db_data:/var/lib/postgresql/data # persistencia de los datos
      - ./db/init.sql:/docker-entrypoint-initdb.d/init.sql:ro
    networks:
      - red_notas
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U $${POSTGRES_USER} -d $${POSTGRES_DB}"]
      interval: 5s
      timeout: 5s
      retries: 10
    # Sin "ports": la base de datos solo es accesible dentro de la red de Docker

  # Contenedor 1: aplicación web (frontend + API)
  app:
    build: ./app
    image: notas-app:1.0.0
    container_name: notas-app
    restart: unless-stopped
    environment:
      APP_PORT: 3000
      DB_HOST: db # NOMBRE DEL SERVICIO, no localhost ni IP
      DB_PORT: 5432
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      POSTGRES_DB: ${POSTGRES_DB}
    ports:
      - "${APP_PUBLIC_PORT:-8080}:3000"
    depends_on:
      db:
        condition: service_healthy
    healthcheck:
      test: ["CMD", "wget", "-qO-", "http://localhost:3000/api/health"]
      interval: 5s
      timeout: 5s
      retries: 10
    networks:
      - red_notas

networks:
  red_notas:
    name: red_notas
    driver: bridge

volumes:
```

```
  db_data:
    name: notas_db_data
```

3.3 .env.example (el archivo .env real no se sube)

```
# Copia este archivo como .env y cambia los valores:  cp .env.example .env
# El archivo .env NO se sube al repositorio (está en .gitignore)
POSTGRES_USER=notas_user
POSTGRES_PASSWORD=cambia_esta_contrasena
POSTGRES_DB=notas_db
APP_PUBLIC_PORT=8080
```

3.4 Conexión a la base de datos en app/server.js (fragmento)

```
// Contenedor 1: aplicación web (frontend + API REST)
// Se conecta al contenedor 2 (PostgreSQL) usando el NOMBRE DEL SERVICIO "db".
const express = require('express');
const path = require('path');
const { Pool } = require('pg');

const PORT = process.env.APP_PORT || 3000;

// Todos los datos de conexión llegan por variables de entorno (archivo .env).
const pool = new Pool({
  host: process.env.DB_HOST, // "db" -> nombre del servicio en docker-compose
  port: Number(process.env.DB_PORT || 5432),
  user: process.env.POSTGRES_USER,
  password: process.env.POSTGRES_PASSWORD,
  database: process.env.POSTGRES_DB,
});
```

4. Evidencias

4.1 Construcción de la imagen (docker build)

Terminal — docker build

```
usuario@docker:~/CICDSFT2$ docker build -t notas-app:1.0.0 ./app
#0 building with "default" instance using docker driver

#1 [internal] load build definition from Dockerfile
#1 transferring dockerfile: 565B done
#1 DONE 0.0s

#2 [internal] load metadata for docker.io/library/node:20.18-alpine
#2 DONE 0.0s

#3 [internal] load .dockerignore
#3 transferring context: 114B done
#3 DONE 0.0s

#4 [internal] load build context
#4 transferring context: 37.76kB done
#4 DONE 0.0s

#5 [1/6] FROM docker.io/library/node:20.18-alpine@sha256:ae06ecd567cad0724419280db6728cf7f4479930d30525bdd6da443b91d7d3
23
#5 resolve docker.io/library/node:20.18-alpine@sha256:ae06ecd567cad0724419280db6728cf7f4479930d30525bdd6da443b91d7d323
0.0s done
#5 DONE 0.0s

#6 [2/6] WORKDIR /usr/src/app
#6 CACHED

#7 [3/6] COPY package.json package-lock.json ./
#7 DONE 0.0s

#8 [4/6] RUN npm ci --omit=dev && npm cache clean --force
#8 1.979
#8 1.979 added 87 packages, and audited 88 packages in 2s
#8 1.979
#8 1.979 17 packages are looking for funding
#8 1.979   run `npm fund` for details
#8 1.980
#8 1.980 found 0 vulnerabilities
#8 DONE 2.4s

#9 [5/6] COPY server.js ./
#9 DONE 0.0s

#10 [6/6] COPY public ./public
#10 DONE 0.0s

#11 exporting to image
#11 exporting layers 0.1s done
#11 exporting manifest sha256:35cc99e67e9d1d47c60a5cd87a5c1c3fdfa9db5f821316354e5d6723baf3d66b done
#11 exporting config sha256:4a5c650028cb0ed9a07219bb1253197e291d56aa00a52426b6b1f050caee1172 done
#11 exporting manifest list sha256:d41b9f1f8d50b563376788e0bc22aef1fb41956272b242dfaff86e0393ca6309 done
#11 naming to docker.io/library/notas-app:1.0.0 done
#11 unpacking to docker.io/library/notas-app:1.0.0 0.1s done
#11 DONE 0.3s
```

Figura 2. Construcción de la imagen `notas-app:1.0.0` sin errores: 6 pasos del Dockerfile, 87 paquetes instalados y 0 vulnerabilidades.

4.2 Levantar los servicios (docker compose up)

Terminal — docker compose up

```
usuario@docker:~/CICDSFT2$ docker compose up -d
 Network red_notas Creating
 Volume notas_db_data Creating
 Volume notas_db_data Created
 Network red_notas Created
 Container notas-db Creating
 Container notas-db Created
 Container notas-app Creating
 Container notas-app Created
 Container notas-db Starting
 Container notas-db Started
 Container notas-db Waiting
 Container notas-db Healthy
 Container notas-app Starting
 Container notas-app Started
 Container notas-app Waiting
 Container notas-db Waiting
 Container notas-db Healthy
 Container notas-app Healthy
```

Figura 3. `docker compose up -d` crea la red `red_notas`, el volumen `notas_db_data` y los dos contenedores. La app arranca solo cuando la base de datos está *Healthy*.

4.3 Contenedores en ejecución (docker ps)

Terminal — docker ps

```
usuario@docker:~/CICDSFT2$ docker ps
CONTAINER ID   IMAGE                 STATUS                    PORTS                    NAMES
a9745841b1c1   notas-app:1.0.0       Up 5 seconds (healthy)    0.0.0.0:8080->3000/tcp   notas-app
7c7b89ec99ae   postgres:16.4-alpine  Up 11 seconds (healthy)   5432/tcp                 notas-db
```

Figura 4. Ambos contenedores en estado *healthy*. Solo la app publica un puerto al host (8080→3000); PostgreSQL (5432) queda interno.

Terminal — logs app

```
usuario@docker:~/CICDSFT2$ docker compose logs app
notas-app  | App escuchando en el puerto 3000; base de datos en host "db"
```

Figura 5. Log de la app: escucha en el puerto 3000 y usa la base de datos en el host `db`.

4.4 Red de Docker (docker network inspect)

Terminal — docker network inspect

```
usuario@docker:~/CICDSFT2$ docker network inspect red_notas   # (campos principales)
[
    {
        "Name": "red_notas",
        "Driver": "bridge",
        "Scope": "local",
        "IPAM": {
            "Config": [
                {
                    "Subnet": "172.18.0.0/16",
                    "Gateway": "172.18.0.1"
                }
            ]
        },
        "Containers": {
            "notas-db": {
                "IPv4Address": "172.18.0.2/16",
                "MacAddress": "5e:1d:bd:8a:57:c6"
            },
            "notas-app": {
                "IPv4Address": "172.18.0.3/16",
                "MacAddress": "6e:88:56:1e:6c:24"
            }
        },
        "Labels": {
            "com.docker.compose.network": "red_notas",
            "com.docker.compose.project": "cicdsft2"
        }
    }
]
```

Figura 6. `docker network inspect red_notas` (campos principales): red bridge con los dos contenedores conectados.

4.5 Prueba de comunicación (ida y vuelta)

Se demuestra que el contenedor app resuelve el nombre del servicio `db`, guarda un dato en PostgreSQL y lo lee de vuelta; el dato también se verifica directamente dentro del contenedor db.

Terminal — prueba de comunicación

```
# 1) Desde el contenedor app, el nombre "db" se resuelve por el DNS interno de Docker
usuario@docker:~/CICDSFT2$ docker exec notas-app getent hosts db
172.18.0.2        db  db

# 2) GUARDAR un dato: la app lo inserta en PostgreSQL
usuario@docker:~/CICDSFT2$ curl -X POST -H "Content-Type: application/json" -d '{"texto":"Hola desde el contenedor ap
p"}' http://localhost:8080/api/notas
{"id":2,"texto":"Hola desde el contenedor app","creado":"2026-11-11T02:29:19.271Z"}

# 3) LEER el dato de vuelta a través de la app
usuario@docker:~/CICDSFT2$ curl http://localhost:8080/api/notas
[
    {
        "id": 2,
        "texto": "Hola desde el contenedor app",
        "creado": "2026-11-11T02:29:19.271Z"
    },
    {
        "id": 1,
        "texto": "Nota inicial creada por init.sql",
        "creado": "2026-11-11T02:29:08.990Z"
    }
]

# 4) Verificar directamente en el contenedor db
usuario@docker:~/CICDSFT2$ docker exec notas-db sh -c 'psql -U "$POSTGRES_USER" -d "$POSTGRES_DB" -c "SELECT * FROM not
as;"'
 id |              texto               |           creado
----+----------------------------------+----------------------------
  1 | Nota inicial creada por init.sql | 2026-11-11 02:29:08.990894
  2 | Hola desde el contenedor app     | 2026-11-11 02:29:19.271628
(2 rows)

# 5) Estado de la conexión
usuario@docker:~/CICDSFT2$ curl http://localhost:8080/api/health
{
    "app": "ok",
    "db": "ok",
    "db_host": "db",
    "db_ip": "172.18.0.2",
    "version": "PostgreSQL 16.4 on x86_64-pc-linux-musl, compiled by gcc (Alpine 13.2.1_git20240309) 13.2.1 20240309, 6
4-bit"
}
```

Figura 7. Prueba de comunicación: resolución DNS de `db`, inserción (POST), lectura (GET), verificación con `psql` y estado de la conexión.

Terminal — persistencia

```
usuario@docker:~/CICDSFT2$ docker compose down && docker compose up -d
usuario@docker:~/CICDSFT2$ curl http://localhost:8080/api/notas   # el volumen notas_db_data conserva los datos
[
    {
        "id": 2,
        "texto": "Hola desde el contenedor app",
        "creado": "2026-11-11T02:29:19.271Z"
    },
    {
        "id": 1,
        "texto": "Nota inicial creada por init.sql",
        "creado": "2026-11-11T02:29:08.990Z"
    }
]
```

Figura 8. Persistencia: después de `docker compose down` y `up`, la nota sigue guardada gracias al volumen.



Figura 9. Frontend de la aplicación en http://localhost:8080: la nota escrita en el navegador se guarda y se lee desde PostgreSQL.

4.6 Integración continua (GitHub Actions)

Ejecución #3 del workflow **Docker CI** sobre el commit `6eab9c1`. Resultado: **SUCCESS**. Enlace: https://github.com/santiagogamboac/CICDSFT2/actions/runs/42946404369

	Paso del job `build-and-test`	**Resultado**
✅	Set up job	success
✅	Descargar el código	success
✅	Crear .env para CI (valores de prueba, no son secretos reales)	success
✅	Validar docker-compose.yml	success
✅	Construir las imágenes	success
✅	Levantar los contenedores	success
✅	Probar comunicación app -> db (guardar y leer)	success
✅	Ver contenedores y logs	success
✅	Apagar	success
✅	Complete job	success

Tabla 1. Pasos del pipeline obtenidos de la API de GitHub Actions. En cada push y pull request el pipeline construye las imágenes, levanta los servicios y verifica que se pueda guardar y leer una nota.

5. Conclusiones

- Docker Compose permite describir en un solo archivo una aplicación de varios servicios (contenedores, red, volúmenes y variables) y levantarla de forma reproducible con un solo comando.
- Dentro de una red definida por el usuario, Docker ofrece un DNS interno que resuelve el nombre de cada servicio; por eso la app se conecta a `db` sin depender de `localhost` ni de IPs que cambian en cada ejecución.
- Los healthchecks junto con `depends_on: condition: service_healthy` evitan que la aplicación arranque antes de que la base de datos esté lista.
- Separar los secretos en un archivo `.env` no versionado, usar `.dockerignore`, fijar versiones de imágenes y ejecutar como usuario sin privilegios son prácticas sencillas que mejoran la seguridad y la reproducibilidad.
- Integrar Docker al pipeline de GitHub Actions permite detectar en cada cambio si las imágenes dejan de construirse o si los servicios dejan de comunicarse; el primer run falló porque faltaba

el `.env.example` en el repositorio, y el pipeline lo detectó de inmediato.

Referencias

Docker Inc. (2024). *Docker Compose overview*. Docker Docs. https://docs.docker.com/compose/

Docker Inc. (2024). *Dockerfile reference*. Docker Docs. https://docs.docker.com/reference/dockerfile/

Docker Inc. (2024). *Networking in Compose*. Docker Docs. https://docs.docker.com/compose/how-tos/networking/

Docker Inc. (2024). *Building best practices*. Docker Docs. https://docs.docker.com/build/building/best-practices/

GitHub Inc. (2024). *Understanding GitHub Actions*. GitHub Docs. https://docs.github.com/en/actions/about-github-actions/understanding-github-actions

Merkel, D. (2014). Docker: Lightweight Linux containers for consistent development and deployment. *Linux Journal*, *2014*(239), Artículo 2.

The PostgreSQL Global Development Group. (2024). *PostgreSQL 16 documentation*. https://www.postgresql.org/docs/16/